Cryptography Through Abstract Algebra: An Interactive Study of Three Transformation Systems

P.V.S. Subramanya Prasad (24MMI0011) Ananya Anand (24MMI0010)
Aditya Datta (24MMI0013) Varun Raaj (24MMI0009)
Pavani Durga C (24MMI0020)

19 October 2026

Abstract

This paper compares three finite systems of reversible transformations: Caesar shifts modulo 26, invertible affine transformations modulo 26, and permutations of four positions (the symmetric group S_4). Putting all three in one group-theoretic framework lets us say exactly where they agree and where they differ. Caesar shifts form the cyclic abelian group C_{26} of order 26. Invertible affine maps $x \mapsto ax + b \pmod{26}$, with $\gcd(a, 26) = 1$, form a non-abelian group of order 312. Four-position permutations form the non-abelian group S_4 of order 24. We go through the group axioms, work out inverse and composition examples, and prove a few subgroup results. The main one is that the Caesar shifts are the normal translation subgroup of the affine group, which is a semidirect product $\mathbb{Z}_{26} \rtimes \mathbb{Z}_{26}^{\times}$.

The interactive resource is the Interactive Abstract Algebra Lab, and its source is on GitHub. We keep software and testing statements apart from the mathematical proofs. The technical dossier we had reports some checks, but it does not show that every deployed feature was tested. The website is an educational demonstration. It is not a secure cryptographic system, and it is not a measured learning intervention.

Keywords: abstract algebra; group theory; Caesar cipher; affine cipher; symmetric group; permutations; modular arithmetic; interactive visualization.

1 Introduction

Reversible transformations are a good place to see composition, identity, and inverse at work. Abstract algebra lets us ask how systems that look the same on screen can still differ mathematically. A group is a set with a binary operation that is closed and associative, has an identity element, and gives every element an inverse. We study three of them: additive shifts of a 26-letter alphabet, invertible affine transformations on the integers modulo 26, and permutations of four positions. Caesar shifts form a cyclic abelian group. The other two are non-abelian. Composing Caesar shifts in either order gives the same result, but in the other two systems the order matters.

We do not claim anything new here: no new cryptographic algorithm, no new theorem in group theory, and no field-wide novelty for the literature we reviewed. We also do not claim that students learn from the website. No study of students using it is documented in the materials we were given, so we have no measured learning gains to report.

Research question. How does abstract algebra help describe and compare Caesar shifts, invertible affine transformations modulo 26, and permutations of four positions, and how can an interactive website make selected operations observable?

2 Related work

The website overlaps with earlier systems in some ways and differs in others. Trigui and Alghazzawi (2012), in "Learning the related mathematics to cryptography by interactive way," built an interactive environment and ran a case study on its usability. Users rated it favorably. The authors do not show a causal link between the environment and what students learned.

Abuzaid, Yuan, Yu, and Chu (2011), in "The design and implementation of a cryptographic education tool," built a teaching tool around visualization and step-by-step interaction. Users can explore key generation, encryption, and decryption visually. The authors describe the tool but, again, show no causal link to student learning.

Veith, Bitzenbauer, and Girnat (2022), in "Exploring learning difficulties in abstract algebra: The case of group theory," studied students' difficulties in introductory group theory using a pre-test/post-test design with the concept inventory for introductory group theory (CI2GT). Their results are why we spell out operation, identity, inverse, and composition explicitly.

We did not find a source that treats all three of our systems together. That is a reason for the comparison, but it does not establish novelty across the field.

3 Mathematical preliminaries and a shared framework

A group $(G, \circ)$ is a set G with a binary operation satisfying four axioms:

1. *Closure:* if $g, h \in G$, then $g \circ h \in G$.
2. *Associativity:* $(g \circ h) \circ k = g \circ (h \circ k)$ for all $g, h, k \in G$.
3. *Identity:* there is an element $e \in G$ such that $e \circ g = g \circ e = g$.
4. *Inverses:* for each $g \in G$ there is $g^{-1} \in G$ with $g \circ g^{-1} = g^{-1} \circ g = e$.

The order of a finite group is its number of elements. The order of an element g is the least positive integer n with $g^n = e$. A group is cyclic if one element generates it, and abelian if its operation is commutative. Function composition $g \circ f$ means apply f first, then g (Dummit & Foote, 2004; Fraleigh, 2003).

For alphabet transformations we encode letters as $A = 0$, $B = 1$, $\ldots$, $Z = 25$ and work modulo 26. For permutations we use the one-based output-position convention. A list $p = [p_1, p_2, p_3, p_4]$ acts on $x = x_1x_2x_3x_4$ by $P_p(x)_i = x_{p_i}$, so output position i receives the input character at position p_i. For example, $[3, 1, 4, 2]$ maps ABCD to CADB. The TypeScript implementation we inspected stores indices zero-based and displays them one-based, so this convention has to be stated outright.

3.1 How the four axioms appear in the three systems

The three systems meet the same requirements in different ways. For Caesar shifts, composition adds the shift parameters mod 26. Closure comes from modular addition, the identity is the zero shift, and the inverse of a shift is its negative. For affine maps, composing two allowed maps gives another affine map whose multiplier is a product of units mod 26, which is again a unit. Section 5 gives the identity and inverse explicitly. For permutations, composing bijections gives a bijection, the identity permutation fixes every position, and the inverse bijection reverses the position mapping. In all three systems, associativity follows from associativity of function composition. These arguments establish the axioms. We do not treat software output as proof.

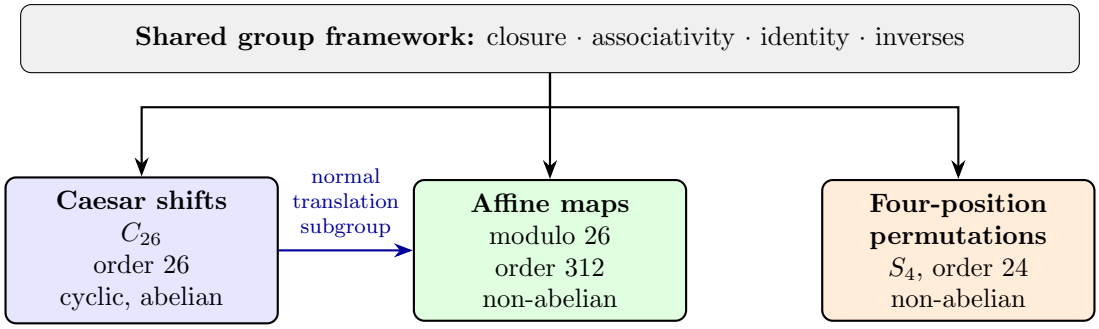


Figure 1: Three concrete transformation groups share the group axioms. Caesar shifts form the normal translation subgroup of the affine group; S_4 is compared as a separate system.

4 Caesar transformations

4.1 Definition and group structure

For each $k \in \mathbb{Z}_{26}$, define $C_k(x) = x + k \pmod{26}$. Applying C_k first and C_j second gives

$$(C_j \circ C_k)(x) = (x + k) + j = x + (k + j) \pmod{26},$$

so $C_j \circ C_k = C_{j+k}$. Closure follows from addition modulo 26, the identity is C_0, and the inverse of C_k is C_{-k}. Associativity follows from associativity of function composition (equivalently, of modular addition). Every shift is a power of C_1, so the group is cyclic and isomorphic to C_{26}. It is abelian because modular addition is commutative.

4.2 Element orders, generators, and subgroups

The order of a shift is

$$\operatorname{ord}(C_k) = \frac{26}{\gcd(k, 26)}, \qquad \gcd(0, 26) = 26.$$

Since $26 = 2 \cdot 13$, the possible element orders are 1, 2, 13, and 26. The identity has order 1, C_{13} has order 2, the twelve nonzero even shifts have order 13, and the twelve shifts relatively prime to 26 have order 26. The generators are C_k for $k \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$, so there are $\varphi(26) = 12$ of them. A cyclic group has exactly one subgroup for each positive divisor of its order, so C_{26} has four subgroups, of orders 1, 2, 13, and 26: $\{C_0\}$, $\{C_0, C_{13}\}$, the subgroup of all even shifts, and the whole group.

4.3 Worked example

With shift $k = 3$, ABC is encoded as $(0, 1, 2)$ and maps to $(3, 4, 5)$, which is DEF. Since $-3 \equiv 23 \pmod{26}$, applying the shift by 23 restores ABC.

5 Invertible affine transformations modulo 26

5.1 Definition and group order

Define $E_{a,b}(x) = ax + b \pmod{26}$, where $a, b \in \mathbb{Z}_{26}$ and $\gcd(a, 26) = 1$. The condition on a is necessary and sufficient for multiplication by a to be invertible modulo 26. There are $\varphi(26) = 12$ allowed multipliers and 26 choices of offset, giving $12 \cdot 26 = 312$ distinct affine maps.

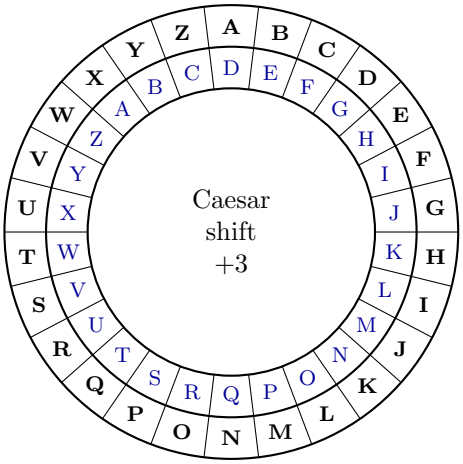


Figure 2: Alphabet wheel for a Caesar shift of +3. Each plaintext letter maps to the ciphertext letter three positions ahead, wrapping from Z to A; for example, A $\mapsto$ D and X $\mapsto$ A.

5.2 Composition, identity, and inverse

Applying $E_{c,d}$ first and $E_{a,b}$ second gives

$$(E_{a,b} \circ E_{c,d})(x) = a(cx + d) + b = (ac)x + (ad + b) \pmod{26},$$

so $E_{a,b} \circ E_{c,d} = E_{ac,ad+b}$, with parameters reduced modulo 26. A product of units is a unit, so ac is still a unit, and that gives closure. The identity is $E_{1,0}$. Solving $y = ax + b \pmod{26}$ for x gives

$$E_{a,b}^{-1}(y) = a^{-1}(y - b) \pmod{26},$$

which is $E_{a^{-1},-a^{-1}b}$. Associativity is inherited from function composition, and the identity and inverse above satisfy the group requirements. So the affine maps form a group under composition.

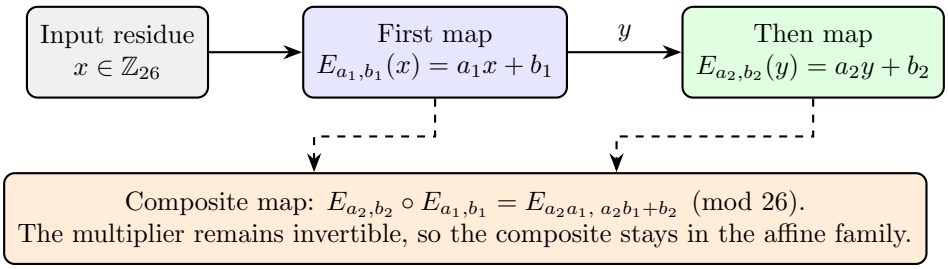


Figure 3: Affine composition keeps transformations within the same group. The order of composition matters, even though closure is preserved.

5.3 Worked inverse example

For $E_{5,8}$, the inverse of 5 modulo 26 is 21, because $5 \cdot 21 = 105 \equiv 1 \pmod{26}$. The inverse offset is $-21 \cdot 8 = -168 \equiv 14 \pmod{26}$. So

$$E_{5,8}^{-1}(y) = 21y + 14 \pmod{26}.$$

Encrypting ABC gives A (0) $\mapsto$ 8 (I), B (1) $\mapsto$ 13 (N), and C (2) $\mapsto$ 18 (S), so the ciphertext is INS. Decrypting gives $21 \cdot 8 + 14 = 182 \equiv 0$, $21 \cdot 13 + 14 = 287 \equiv 1$, and $21 \cdot 18 + 14 = 392 \equiv 2 \pmod{26}$, which restores ABC.

5.4 Non-commutativity

Let $f = E_{3,1}$ and $g = E_{1,5}$. Applying f first and g second gives

$$(g \circ f)(x) = (3x + 1) + 5 = 3x + 6 \pmod{26},$$

while applying g first and f second gives

$$(f \circ g)(x) = 3(x + 5) + 1 = 3x + 16 \pmod{26}.$$

At $x = 0$ the outputs are 6 and 16. The maps differ, so the affine group is non-abelian.

5.5 Translation subgroup and semidirect product

The translations $T_b = E_{1,b}$ form a subgroup of order 26 isomorphic to $(\mathbb{Z}_{26}, +)$. They are exactly the Caesar shifts. The multiplier maps $M_a = E_{a,0}$ form a subgroup isomorphic to the unit group $\mathbb{Z}_{26}^{\times}$, of order 12. Every affine map factors uniquely as $E_{a,b} = T_b \circ M_a$. Conjugation gives

$$M_a \circ T_b \circ M_a^{-1} = T_{ab}.$$

So the translation subgroup is normal, and the affine group is the semidirect product $\mathbb{Z}_{26} \rtimes \mathbb{Z}_{26}^{\times}$, with each unit acting on the additive group by multiplication. In other words, an affine map is a translation combined with an invertible rescaling, and the rescalings act on the translations by conjugation (Dummit & Foote, 2004).

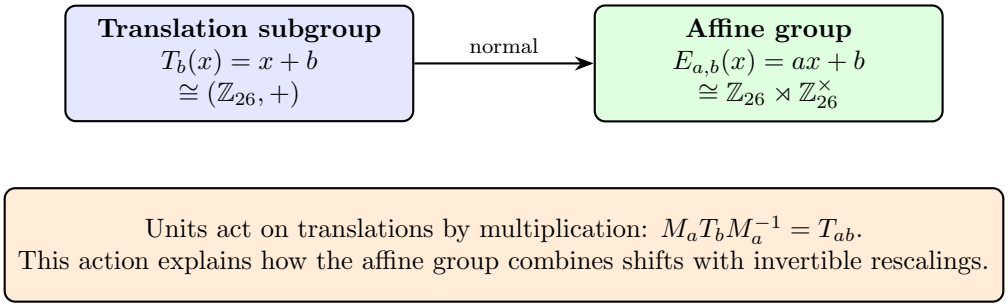


Figure 4: The affine group extends Caesar translations by adding invertible multipliers. The semidirect product records this interaction.

The centre is $\{E_{1,0}, E_{1,13}\}$. To see why, suppose $E_{a,b}$ commutes with every translation $E_{1,c}$. Comparing the two compositions for $c = 1$ forces $a \equiv 1 \pmod{26}$. A translation $E_{1,b}$ commutes with every multiplier $E_{u,0}$ exactly when $(u - 1)b \equiv 0 \pmod{26}$ for every unit u. Taking $u = 3$ gives $2b \equiv 0 \pmod{26}$, so b is 0 or 13. Conversely, every unit u is odd, so $(u-1)\cdot 13$ is divisible by 26, which confirms that $E_{1,13}$ commutes with all multipliers and translations. These are selected structural results, not a full classification of element orders or subgroups. The translation subgroup does show directly how the Caesar system sits inside a larger non-abelian group.

6 Four-position permutations and S_4

6.1 The symmetric group

The symmetric group S_4 consists of all bijections of four positions, so it has $4! = 24$ elements. Composition of bijections is closed and associative, the identity bijection fixes every position, and every bijection has an inverse. The possible element orders are 1, 2, 3, and 4. By cycle type,

S_4 has the identity (1), transpositions (6), double transpositions (3), 3-cycles (8), and 4-cycles (6), which add up to 24.

The alternating group A_4 consists of the 12 even permutations. The Klein four-group

$$V_4 = \{e, (12)(34), (13)(24), (14)(23)\}$$

is a subgroup of A_4 and is normal in S_4. S_4 is non-abelian because the transpositions (12) and (23) give different products in opposite orders.

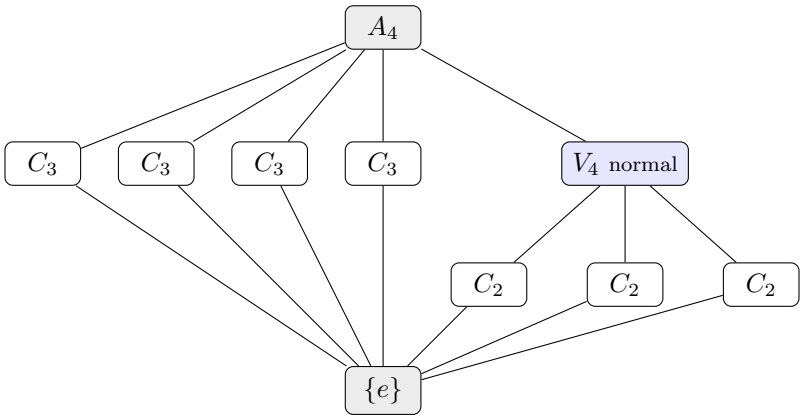


Figure 5: A subgroup-lattice view of A_4. It contains four subgroups isomorphic to C_3 and the normal Klein four-group V_4, whose three non-identity elements generate three subgroups of order 2. All these proper subgroups meet the trivial subgroup at the bottom.

6.2 Output-position convention and worked composition

Let $p = [3, 1, 4, 2]$ and $q = [2, 1, 4, 3]$. Under the convention $P_p(x)_i = x_{p_i}$, applying q first and then p gives composite entries $(p \star q)_i = q_{p_i}$. On ABCD, q gives BADC, and then p gives DBCA, so the composite list is $[4, 2, 3, 1]$. In the other order, p gives CADB and then q gives ACBD, with composite list $[1, 3, 2, 4]$. DBCA and ACBD are different, so these transformations do not commute.

The inverse of $[3, 1, 4, 2]$ is $[2, 4, 1, 3]$. Applying the original list to ABCD gives CADB, and applying the inverse list to CADB gives back ABCD. The inverse reverses the position correspondence.

The TypeScript source we inspected stores permutations as zero-based lists and composes them with `first.map(index => second[index])`, which matches the one-based formula above. The separate Python verifier uses the same formula. That is an observation about the source code. It does not prove that every website interaction has been exercised.

7 Comparative analysis

All three systems satisfy the same group axioms, but their orders (26, 312, and 24) differ. Isomorphic finite groups must have equal order, so the three groups are pairwise non-isomorphic. The Caesar shifts sit inside the affine group as its normal translation subgroup. S_4 is a separate permutation system, and we do not assert any direct structural relationship between it and the affine group. What the comparison shows is that one abstract framework covers different transformation rules and different internal structures.

Property	Caesar shifts	Affine maps modulo 26	Four-position permutations
Group	C_{26}	$\mathbb{Z}_{26} \rtimes \mathbb{Z}_{26}^{\times}$	S_4
Order	26	312	24
Operation	Composition (shift addition)	Composition of $ax + b$ maps	Composition of permutations
Identity	C_0	$E_{1,0}$	$[1, 2, 3, 4]$
Inverse	C_{-k}	$E_{a^{-1},-a^{-1}b}$	Inverse permutation
Abelian?	Yes	No	No
Cyclic?	Yes	No	No
Selected structure	12 generators; four subgroups	Normal translation subgroup; centre has two elements	Contains A_4 and normal V_4
Role	Alphabet substitution by shift	Alphabet substitution by multiplier and offset	Rearranges four-character blocks

Table 1: The three finite transformation groups side by side.

8 Methodology and implementation

8.1 Mathematical workflow

For each transformation we define it, derive its operation and inverse, compute the group order and selected element orders, and build explicit examples of inverse restoration and non-commutativity. General claims rest on algebraic arguments, and the worked examples illustrate particular calculations. Software output is never a substitute for a proof.

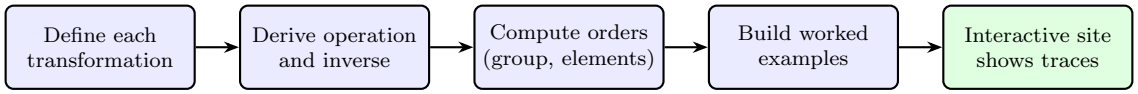


Figure 6: Mathematical workflow. Proofs come from the algebraic arguments; the website illustrates them.

8.2 Repository and technology inspected

The public repository for this audit is `https://github.com/PVSSP656/final-algebra-lab`. The inspection record gives the default branch as `main`, at commit `9496d260fbaf83b69b0fe83c1b243d224cf9a3df` (18 October 2026), with the merge message "Fix S4 permutation decryption workflow." The files inspected were `README.md`, `package.json`, `src/math/engine.ts`, `src/App.tsx`, and `scripts/verify_math.py`. Nothing in the repository was changed during that inspection.

The stack is React 19 with TypeScript, built with Vite and configured through the React Vite plugin. The package declares KaTeX, lucide-react, and Motion. The application has views labelled Overview, Foundations, Group Explorer, The Lab, Algebraic Structure, Experiments, The Team, and Research Log. The main components are CipherWheel, GroupCipherCards, LabCalculations, CommonAlgebraicStructure, ExperimentsLab, and TransformationChamber. Firebase Hosting configuration is present, and the README calls the site an educational tool, not a secure messaging service.

The shared typed engine implements Caesar arithmetic, affine arithmetic and inverses, permutation application, composition, and inversion, trace generation, order calculation, and letter-frequency analysis. The Python verifier is a separate implementation that uses assertions to check selected laws and examples. The package scripts are `dev`, `build`, `lint`, `verify`, and `preview`, and `verify` runs `python scripts/verify_math.py`. The `package.json` we inspected has no test script, and we found no standard TypeScript test-framework configuration in the tree. So the Python verifier should not be described as a TypeScript unit-test suite.

8.3 Evidence classes and test status

The technical QA dossier records that build, lint, and the Python verifier passed on a local checkout at commit `91fb67b13a47bf62ca2e7dedc348ad69a6863e66`. The build also warned that an output chunk was over 500 kB. These results come from the dossier, and we did not rerun the commands for this revision. The verifier prints "All Caesar, affine, S4, and frequency verification checks passed successfully." Its assertions exercise Python functions. It does not import or run the TypeScript engine.

The dossier records an earlier browser failure in S4 decryption. It then reports that after the project lead fixed it, the regression case CADB $\rightarrow$ ABCD with key $[3, 1, 4, 2]$ passed, as did inverse-key encryption and the incomplete-block case ABCDE $\rightarrow$ CADBE. These are manual browser observations, and the dossier has no screenshots of them. It does not say which revision was tested after the fix, and it has no matching build, lint, and verify run for that revision. The current GitHub branch head has a later commit titled "Fix S4 permutation decryption workflow," but that alone does not show the deployed site was built from it.

During the earlier audit we tried to open the live site (`https://abstract-algebra-lab.web.app/`) with the page-opening tool, and fetching was reported as disabled. A direct HTTP request also failed because DNS and network resolution were unavailable in that environment. So we did not independently inspect the live page's appearance, runtime behaviour, or screenshots. That is a limit on our access and not evidence that the site is broken. The team has declared the known bugs fixed, and we take that as given. The earlier defect is treated as historical, with the post-fix case reported as passing.

8.4 Methodological boundary

No controlled educational study, participant sample, survey data, learning-gain estimate, or inferential statistics for this website are documented. We cannot carry findings from the cited literature over to this project as measured outcomes. The paper therefore makes no causal claim that the website improves learning.

9 Limitations and cryptographic scope

The literature review covers three sources and is not systematic, so it cannot support a claim of novelty across the field. No controlled evaluation of learning with this website is documented. The software evidence is tied to specific revisions: we did not fetch the live site in the earlier audit, and the dossier does not give the exact commit used for the post-fix browser tests. The Python verifier is independent of the TypeScript engine, and we found no automated TypeScript test suite in the public tree.

Caesar and affine substitution ciphers have small key spaces and keep letter-frequency patterns, apart from relabelling. A fixed-block position permutation changes the order of characters

within each complete block but leaves the characters themselves alone. These systems are useful for demonstrating finite algebraic structure. They are not suitable for protecting confidential communication (Stinson, 2005).

10 Conclusion and future work

Group theory puts three different reversible transformations into one framework. Caesar shifts form C_{26}, a cyclic abelian group with 26 elements, 12 generators, and four subgroups. Invertible affine transformations modulo 26 form a non-abelian group of order 312. The Caesar shifts are its normal translation subgroup, its structure is the semidirect product $\mathbb{Z}_{26} \rtimes \mathbb{Z}_{26}^{\times}$, and its centre is $\{E_{1,0}, E_{1,13}\}$. Four-position permutations form the non-abelian group S_4 of order 24, where the order of composition matters. The worked calculations illustrate inverse restoration and non-commutativity, and the different group orders prove the three groups are pairwise non-isomorphic.

The website lets readers explore selected transformations and traces. The live project and the source repository are linked separately so readers can try the demonstration and read the code. The evidence we have does not support claims that the site is free of bugs, that a release was verified, or that it improves learning. Future work could record a test run against an explicitly named post-fix commit, add automated TypeScript regression tests tied to the conventions used here, capture real screenshots of the deployed site, and run a separately designed study if educational effectiveness is to be evaluated.

References

Abuzaid, A., Yuan, X., Yu, H., & Chu, B. (2011). The design and implementation of a cryptographic education tool. In *Proceedings of the 3rd International Conference on Computer Supported Education (CSEDU 2011)*, Vol. 1, pp. 193–198. SciTePress. https://doi.org/10.5220/0003301301930198

Dummit, D. S., & Foote, R. M. (2004). *Abstract Algebra* (3rd ed.). John Wiley & Sons. Relevant background includes group axioms, symmetric groups, cyclic groups, homomorphisms, and semidirect products.

Fraleigh, J. B. (2003). *A First Course in Abstract Algebra* (7th ed.). Addison-Wesley.

PVSSP656. (2026). *Final Algebra Lab* [Computer software]. GitHub repository, branch main, commit `9496d260fbaf83b69b0fe83c1b243d224cf9a3df`. https://github.com/PVSSP656/final-algebra-lab. Files recorded as examined: `README.md`, `package.json`, `src/math/engine.ts`, `src/App.tsx`, and `scripts/verify_math.py`.

Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman & Hall/CRC. Edition-level page locators are omitted because the project files do not identify pages consulted.

Trigui, M. S., & Alghazzawi, D. M. (2012). Learning the related mathematics to cryptography by interactive way. *International Journal of Modern Education and Computer Science*, 4(2), 8–14. https://doi.org/10.5815/ijmecs.2012.02.02

Veith, J. M., Bitzenbauer, P., & Girnat, B. (2022). Exploring learning difficulties in abstract algebra: The case of group theory. *Education Sciences*, 12(8), Article 516. https://doi.org/10.3390/educsci12080516